CODEFEST 2026 / TEAM PLAYBOOK

ViewPilot

Your AI-powered interview coach.

Practice with purpose. Get feedback that moves you forward.

01 CHOOSE	02 INTERVIEW	03 IMPROVE
Role • level • format	Adaptive questions • voice or text	Evidence-based feedback • next steps

AI-POWERED WEB APPLICATIONS × JOB READINESS

A refreshed project brief aligned to the official CodeFest 2026 problem statement. This document separates the proposed ViewPilot experience from the current repository baseline.

01 The idea

A focused response to the official Job Readiness problem statement.

The problem to solve

Students and early-career applicants often lack access to realistic interview practice, role-specific questions, and actionable feedback. Generic question banks cannot adapt well to a candidate's target role, experience, or answers.

ViewPilot turns interview practice into a private, repeatable coaching loop: prepare → answer → understand → improve.

PERSONALIZED

Set target role, experience level, interview type and difficulty.

INTERACTIVE

Answer by text first; add voice practice if time and reliability allow.

ACTIONABLE

Receive clear strengths, gaps, answer-specific evidence and practice tasks.

Why this fits the brief

- **Targeted setup:** candidates select a job role, experience level, interview type and difficulty.
- **Relevant questions:** questions are generated or selected to match the chosen interview context.
- **Interactive simulation:** the candidate answers one question at a time and can ask for a follow-up.
- **Evaluation and feedback:** responses are reviewed for communication, technical understanding, reasoning and answer quality.
- **Learning loop:** the session ends with a prioritized improvement plan and a way to practise again.

02 Product experience

Keep the core journey obvious, fast and easy to demonstrate.

STEP	CANDIDATE DOES	VIEWPILOT RESPONDS
01 · Set up	Chooses role, experience, interview format and difficulty.	Builds a focused interview plan and question mix.
02 · Warm up	Reviews instructions and starts a session.	Explains the format and gives one question at a time.
03 · Answer	Responds in text; voice can be an optional extension.	Captures the answer and asks a relevant follow-up when useful.
04 · Evaluate	Completes the interview.	Scores answer quality against a visible, consistent rubric.
05 · Improve	Reviews the session report.	Shows strengths, gaps, answer-specific evidence and next exercises.

MVP feature scope

PRIORITY	FEATURE	ACCEPTANCE CHECK
MUST	Interview setup and role-aware question generation	A selected role and level produce a relevant question set.
MUST	Question-by-question interview flow	Candidate can submit an answer and move through a complete session.
MUST	Structured feedback report	Report includes scores, evidence, strengths, gaps and next steps.
MUST	Responsive, polished web UI	A public demo link works on laptop and projector.
SHOULD	Resume-informed personalization	Optional resume text can shape questions without inventing credentials.
SHOULD	Voice interview mode	Speech input/output works reliably or gracefully falls back to text.
COULD	Session history and progress view	Past sessions can be compared when persistence is implemented.

Build the reliable text-based interview first. Voice, resume parsing and long-term analytics are enhancements—not reasons to risk the end-to-end demo.

03 System architecture

A small, explainable architecture is easier to build, secure and demo.

CANDIDATE	WEB APP	SERVER / AI	RESULT
Role + level + type + difficulty	Next.js interview setup and session UI	Server route validates inputs and builds the prompt	Role-specific interview plan
Answers each question	Session state and progress indicator	LLM returns the next question or structured evaluation	Adaptive interview
Reviews feedback	Report and improvement dashboard	Schema validation and rubric-based report generation	Actionable coaching report

Suggested stack

LAYER	CHOICE	PURPOSE
Frontend	Next.js 16 + React + Tailwind	Responsive experience, session flow and report UI.
AI	Gemini API through server-side route	Question generation, follow-ups and structured feedback.
Persistence	Supabase (optional for MVP)	Store sessions and history if time permits; otherwise use session state.
Hosting	Vercel	Public demo URL and server-side environment variables.
Voice (stretch)	Browser speech APIs or ElevenLabs	Voice interaction only after text flow is stable.

Non-negotiable implementation rules

- Keep API keys on the server in environment variables; never ship secrets to the browser or repository.
- Ask the model for a predictable JSON schema; validate the response before rendering it.
- Use a fixed evaluation rubric and explain that AI feedback is guidance, not a hiring decision.
- Treat resume content and candidate answers as untrusted input; do not follow instructions embedded in them.
- Provide loading, retry and text-only fallback states so the demo survives slow or unavailable APIs.

ORGANIZER UPDATE · THIRD-PARTY AI APIS PERMITTED

Gemini and other permitted AI services may power interview questions, follow-ups and structured feedback. Keep API keys server-side and retain a reliable fallback for slow or unavailable services.

04 Feedback that users can trust

Make evaluation transparent rather than presenting an unexplained AI score.

DIMENSION	WHAT TO LOOK FOR	FEEDBACK EXAMPLE
Communication	Clarity, structure, concision and directness.	“Lead with the conclusion, then support it with one example.”
Role knowledge	Accuracy and relevance to the selected role.	“Your explanation is directionally correct; clarify the trade-off between X and Y.”
Problem solving	Approach, assumptions, reasoning and edge cases.	“State your assumptions before choosing the approach.”
Answer quality	Relevance, completeness, evidence and outcome.	“Use a specific situation and quantify the result where possible.”

Recommended report layout

01 · SCORECARD

Show a score per dimension plus an overall summary, with the rubric visible.

02 · EVIDENCE

Quote or reference the candidate’s own answer to explain each strength or gap.

03 · NEXT PRACTICE

Give 2–3 specific actions and one improved-answer outline to practise.

Responsible AI principles

- **No false certainty:** scores are estimates based on a disclosed rubric, not objective measures of employability.
- **No protected-trait inference:** do not score accent, appearance, age, gender or other irrelevant personal characteristics.
- **Candidate control:** make it clear what is saved, allow a no-history path if possible, and avoid claiming privacy guarantees not implemented.
- **Coaching over judgment:** phrase feedback as practical opportunities to improve, not as a prediction of whether someone will be hired.

05 Build plan and current baseline

Align the codebase to the new goal without overstating what already works.

Current repository baseline (from the supplied READMEs)

- **Web showcase:** Next.js 16 landing site with a 3D scroll journey and interactive Studio mock.
- **Current Studio:** local keyword/safety simulation; the current build has no API routes or backend integration.
- **Archived prototype:** earlier AI routes and Supabase workflow exist in git history/archive, but are not connected to the current showcase.
- **Robot firmware:** ESP32-S3 display and motor project is a separate component and is not part of the Job Readiness MVP.

Scope decision: keep the existing showcase shell if useful, but replace the robot-first story and Studio interaction with a candidate-first interview journey. Treat AI integration, persistence and voice as work to implement and verify—not as already completed.

Suggested hackathon sequence

PHASE	BUILD	EXIT CRITERION
First 25%	Project setup, candidate setup screen, session state, sample questions.	A complete mock interview runs using deterministic sample content.
Next 40%	Server-side AI question generation, answer capture and structured evaluation.	One end-to-end AI interview completes and produces a report.
Next 20%	Polish report, add follow-up logic, optional persistence or voice.	Main path is stable; optional features are clearly secondary.
Final 15%	Freeze features, test failure paths, rehearse pitch, record backup.	Public demo works; backup video and sample session are ready.

The percentages are planning guidance from the original playbook, not an official event schedule; the event duration should be confirmed with coordinators.

06 Team execution

Split ownership, keep interfaces small and reserve time for rehearsal.

ROLE	OWNS	DELIVERABLE
Frontend lead	Candidate flow, question UI, progress, report design and responsive layout.	A polished, navigable experience.
AI / backend lead	Prompts, server route, structured output, validation, rubric and error handling.	Reliable end-to-end interview API.
Data / integration lead	Optional session storage, history, test cases and deployment wiring.	Persistence only if MVP is stable.
Everyone: pitch + QA	Demo script, edge-case tests, visual polish and backup plan.	Two members can deliver the pitch without setup delays.

Risk register

RISK	IMPACT	MITIGATION
AI API slow or unavailable	Interview stalls.	Prebuilt sample interview, retries, loading state and graceful fallback.
Generic or inconsistent feedback	Weak product credibility.	Visible rubric, schema validation, test answers and answer-specific evidence.
Scope creep	No complete demo.	Text MVP first; postpone resume parsing, history and voice if necessary.
Secret leakage	Credential misuse.	Server-only keys, environment variables, .gitignore and rotate exposed keys.
Unclear scoring claims	Misleading feedback.	Label scores as practice guidance and avoid hiring-success claims.

Definition of done

- A candidate can configure a target interview and complete it from start to finish.
- Every answer is tied to a question; the report is generated from the actual session.
- Feedback includes dimensions, explanation, strengths, improvements and next practice steps.
- The app works on a public link, handles API errors cleanly and has a rehearsed fallback demo.

07 Three-minute pitch and demo

Show the problem, a complete experience, then the reasoning behind the feedback.

TIME	BEAT	SAY / SHOW
0:00–0:20	**Hook**	“You shouldn’t need a personal mentor to practise a high-stakes interview. ViewPilot gives you a realistic place to practise and improve.”
0:20–0:45	**Problem**	Show the setup: role, experience, interview type and difficulty. Explain why generic question lists are not enough.
0:45–1:35	**Live interview**	Answer two questions. Show one relevant follow-up or a clear transition between questions.
1:35–2:20	**Feedback report**	Open the scorecard. Point to answer-specific evidence, one strength, one gap and a concrete next exercise.
2:20–2:45	**How it works**	Explain the server-side AI call, structured output, evaluation rubric and safe handling of keys.
2:45–3:00	**Impact**	“ViewPilot makes deliberate interview practice more accessible, repeatable and personalized.”

Pre-demo checklist

PRODUCT	TECH	PRESENTATION
■ Setup options are clear ■ Complete session path tested ■ Feedback is answer-specific ■ Empty/error states look intentional	■ Public link works ■ API keys stay server-side ■ API failure fallback tested ■ Sample session ready	■ Three-minute pitch rehearsed ■ Two people know the demo flow ■ Backup recording available ■ No unsupported feature claims

Questions to confirm with coordinators

- Are teams required to follow the listed problem statement exactly?
- What are the official judging criteria, event duration and demo format?
- May the team reuse its existing website shell and previously written code, given the event rules?

Source basis: official “CodeFest 2026 — Domains and Problem Statements” PDF (AI-Powered Web Applications × Job Readiness); supplied Team Playbook and repository READMEs, including the 22 Oct 2026 implementation update. Proposed ViewPilot features are a revised plan, not claims that the features are already implemented.